

CENG 421 - Project

Nadir Can Değirmendere

Multi-Client Chat Application Over TCP

Scope:

This project aims to implement a multi-client chat application using TCP sockets, where multiple clients (users) can connect to a single server simultaneously to exchange messages. The server accepts new connections, assigns a communication thread for each client, and broadcasts any received message to all connected clients. The client side receives user input, sends it to the server, and displays incoming messages from the server.

The scope includes:

- A server application (server.c)
- A client application (client.c)
- A multi-user chat infrastructure over TCP
- Basic features such as username handling, listing online users (with the LIST command), and exiting (exit command)

System Architecture

- Server
 - A main thread uses `accept()` to listen for new client connections.
 - For each new client, the server spawns a separate thread that handles incoming messages from that specific client.
 - Shared arrays (protected by a mutex) store the sockets and usernames of connected clients.
 - The very first message received from a client is taken as that client's username; subsequent messages are either commands (LIST, exit) or regular chat messages.
- Client
 - Connects to the server via a single socket.
 - Right after the connection is established, the client sends its username as the very first message.
 - Afterwards, every line typed by the user is sent to the server (could be LIST, exit, or a normal message).
 - Simultaneously, another thread listens for messages from the server and prints them to the screen.

This architecture enables multiple clients (up to a defined maximum 5) to chat with each other through the server.

Protocol

1. Connection Setup

- The client calls `connect()` to the server's IP and port.
- Once connected, the client immediately sends its username as the first line.

2. Messaging Phase

- The client forwards every user input to the server.
- The server identifies the first incoming line as the "username," and subsequent lines as either normal messages or commands.
- The server broadcasts each normal message to all connected clients in the format `username: message`.

3. Commands

- `LIST`: The server responds with a list of all connected clients, including their IP addresses and usernames, but only sends this list back to the requesting client.
- `exit`: The client disconnects by sending `exit`. The server broadcasts a message like `---- username has left the chat ----`, and removes the user from the list of connected clients.

Server Functions

- `socket()` and `bind()`: Create a server socket and bind it to the specified port.
- `listen()`: Wait for incoming client connections.
- `accept()`: Accept a new client connection and return a socket descriptor for that client.
- `receive_message()`: Runs in a thread dedicated to a single client, reads incoming data:
 - The first line is taken as the username.
 - Subsequent lines are either commands (`LIST`, `exit`) or normal chat messages.
 - If `exit` is received, the client is removed from the list, and a departure message is broadcast.
- `send_message()`: Forwards the received messages to all connected clients (broadcast).
- `send_client_list()`: Sends the list of connected clients (IPs and usernames) back to the requesting client when `LIST` is received.

Client Functions

- `connect()`: Connects to the server using the given IP and port.
- `Send Username`: Immediately after the connection is established, the client sends its username as the first line.
- `send_message()`: Sends the user's typed input to the server (whether it's a command or a normal message).
- `receive_message()`: Listens for messages from the server in a separate thread and displays them. If the server disconnects, the client also exits.

File Structure:

Ad	Değiştirme tarihi	Tür	Boyut
client	17.01.2025 23:14	Dosya	17 KB
client	17.01.2025 23:20	C Kaynak Dosyası	3 KB
Makefile	17.01.2025 21:32	Dosya	1 KB
server	17.01.2025 23:14	Dosya	18 KB
server	17.01.2025 23:19	C Kaynak Dosyası	6 KB

Outputs: (Since I didn't have Linux on my computer, I proceeded by running my project on the computer of my friend named Mevlüt.)

```
mevlut@Mevlut:/mnt/c/Users/mevlu$ ./client 127.0.0.1 8080 Nadir
Group chat application connected. (Type 'exit' to quit, 'LIST' to see who is connected)
---- Nadir has joined the chat ----
---- Ahmet has joined the chat ----
Ahmet: Hi everyone!
Hi to you
Nadir: Hi to you
```

```
mevlut@Mevlut:/mnt/c/Users/mevlu$ ./client 127.0.0.1 8080 Ahmet
Group chat application connected. (Type 'exit' to quit, 'LIST' to see who is connected)
---- Ahmet has joined the chat ----
Hi everyone!
Ahmet: Hi everyone!
Nadir: Hi to you
|
```

```
LIST
Connected clients:
[127.0.0.1] Nadir
[127.0.0.1] Ahmet
```

```
exit
mevlut@Mevlut:/mnt/c/Users/mevlu$
```

```
mevlut@Mevlut:/mnt/c/Users/mevlu$ ./server 8080
Group chat server started on port 8080
Connected client IP: 127.0.0.1
Connected client IP: 127.0.0.1
```

DIFFICULTIES:

Message Format: Distinguishing the very first message as a “username” and subsequent lines as commands or regular chat messages can be tricky.

Command vs. Message Parsing: Correctly detecting and handling LIST, exit, or other custom commands within the incoming data.

FUTURES:

Encryption: Add TLS/SSL support to secure data transmission over TCP.

Channel/Room Support: Allow multiple “rooms” or “channels” for separate discussions.

GUI Client: Implement a graphical interface (e.g., using Qt or JavaFX) instead of a terminal-based client.

Scalability: Move to non-blocking I/O with select(), epoll(), or similar techniques to handle a higher number of concurrent connections efficiently.

REFERENCES:

-The Definitive Guide to Linux Network Programming

K. Davis, J. Turner, N. Yocom.